



Universidad Veracruzana

Modelos en Django

Creación de formulario a partir de un modelo

Programación segura

Dr. Juan Manuel Gutiérrez Méndez

Propósito de la sesión

Construir la segunda versión incremental del proyecto Django incorporando el modelo `Bien` y un formulario básico conectado a base de datos.

En esta versión se agrega persistencia de información, pero todavía no se incorporan validaciones personalizadas ni capa de seguridad.

Al finalizar, el estudiante deberá poder:

- explicar qué es un modelo en Django;
- crear el modelo `Bien` ;
- generar y aplicar migraciones;
- registrar el modelo en el administrador;
- construir un formulario basado en el modelo;
- guardar registros en SQLite;
- mostrar los bienes registrados.

Ruta incremental del proyecto

Versión 1

Autenticación, rutas, login y pantallas protegidas.

Versión 2

Modelo Bien, migraciones, administrador y formulario básico.

Versión 3

Validaciones personalizadas y capa de seguridad reutilizable.

El enfoque incremental permite probar primero acceso, después persistencia y finalmente controles de validación más completos.

¿Qué es un modelo en Django?

Un **modelo** es una clase de Python que representa una entidad del dominio y su estructura de datos.

En Django, el modelo permite definir:

- campos;
- tipos de datos;
- restricciones básicas;
- relaciones;
- representación del objeto;
- estructura de la tabla en la base de datos.

El modelo es el punto donde Django conecta el código Python con la estructura persistente de la base de datos.

Modelo y base de datos

Cuando se crea un modelo, Django puede generar una tabla en la base de datos mediante migraciones.

models.py

Define la clase Python.

makemigrations

Genera el archivo de migración.

migrate

Aplica cambios en la base de datos.

SQLite

Almacena los registros.

El modelo no crea la tabla por sí solo. La tabla se crea cuando se generan y aplican migraciones.

Entidad Bien

Para esta versión se trabajará con una sola entidad: Bien .

Campo	Tipo esperado	Propósito
identificador	Texto	Clave funcional del bien.
descripcion	Texto	Describe el bien registrado.
marca	Texto	Indica la marca del bien.
valor	Decimal	Representa el valor monetario.
estatus	Selección	Indica condición general del bien.
creado_en	Fecha y hora	Registra cuándo fue creado.
actualizado_en	Fecha y hora	Registra la última modificación.

Paso 1: crear el modelo Bien

Editar el archivo:

```
apps/bienes/models.py
```

Contenido:

```
from django.db import models

class Bien(models.Model):
    ESTATUS_CHOICES = [
        ("bueno", "Bueno"),
        ("regular", "Regular"),
        ("malo", "Malo"),
    ]

    identificador = models.CharField(max_length=20, unique=True)
    descripcion = models.CharField(max_length=150)
    marca = models.CharField(max_length=80)
    valor = models.DecimalField(max_digits=10, decimal_places=2)
    estatus = models.CharField(max_length=10, choices=ESTATUS_CHOICES)

    creado_en = models.DateTimeField(auto_now_add=True)
    actualizado_en = models.DateTimeField(auto_now=True)

    def __str__(self):
        return self.identificador
```


Lectura del modelo

Elemento	Explicación
<code>models.Model</code>	Clase base que permite a Django tratar <code>Bien</code> como modelo.
<code>CharField</code>	Campo de texto con longitud máxima.
<code>DecimalField</code>	Campo numérico decimal para valores monetarios.
<code>choices</code>	Limita opciones visibles para un campo.
<code>unique=True</code>	Evita repetir el identificador.
<code>auto_now_add=True</code>	Guarda la fecha de creación.
<code>auto_now=True</code>	Actualiza la fecha de modificación.
<code>__str__()</code>	Define cómo se representa el objeto como texto.

Estas restricciones son estructurales. Las validaciones personalizadas del dominio se agregarán en la siguiente versión.

Paso 2: generar migración

Ejecutar:

```
python manage.py makemigrations
```

Resultado esperado:

```
Migrations for 'bienes':  
  apps/bienes/migrations/0001_initial.py  
    + Create model Bien
```

La migración es un archivo que describe cómo debe cambiar la base de datos para reflejar el modelo.

Paso 3: aplicar migración

Ejecutar:

```
python manage.py migrate
```

Django aplicará la migración y creará la tabla correspondiente.

```
Applying bienes.0001_initial... OK
```

Después de aplicar la migración, la base de datos ya puede almacenar registros de la entidad Bien.

Modelo y formulario

Django permite crear formularios a partir de modelos mediante `ModelForm`.

Model

Define campos y estructura de datos.

ModelForm

Construye formulario con base en el modelo.

View

Procesa petición y decide guardar o mostrar errores.

Template

Presenta los campos del formulario.

El formulario se apoya en el modelo para saber qué campos debe mostrar y cómo convertir los datos recibidos.

Paso 4: crear el formulario BienForm

Crear o editar:

```
apps/bienes/forms.py
```

Contenido:

```
from django import forms
from .models import Bien

class BienForm(forms.ModelForm):
    class Meta:
        model = Bien
        fields = [
            "identificador",
            "descripcion",
            "marca",
            "valor",
            "estatus",
        ]
```

En esta versión el formulario usa las validaciones básicas derivadas del modelo. Aún no se agregan métodos `clean\_`.

¿Cómo influye el modelo en el formulario?

Definición en el modelo	Efecto en el formulario
<code>CharField(max_length=20)</code>	Genera un campo de texto con longitud máxima.
<code>DecimalField(...)</code>	Genera entrada numérica decimal.
<code>choices=ESTATUS_CHOICES</code>	Genera una lista de opciones.
<code>unique=True</code>	Evita duplicidad al guardar.
Campos incluidos en <code>fields</code>	Determinan qué campos aparecen en el formulario.

El `ModelForm` evita duplicar la definición de campos: toma como referencia el modelo.

Paso 7: actualizar la vista de listado

Editar:

```
apps/bienes/views.py
```


Contenido:

```
from django.contrib.auth.decorators import login_required
from django.shortcuts import render, redirect

from .forms import BienForm
from .models import Bien

@login_required
def lista_bienes(request):
    bienes = Bien.objects.all().order_by("identificador")
    return render(request, "bienes/lista.html", {"bienes": bienes})
```

La vista consulta la base de datos mediante el modelo y envía los registros al template mediante el contexto.

Paso 8: actualizar la vista de creación

Completar en `apps/bienes/views.py` .

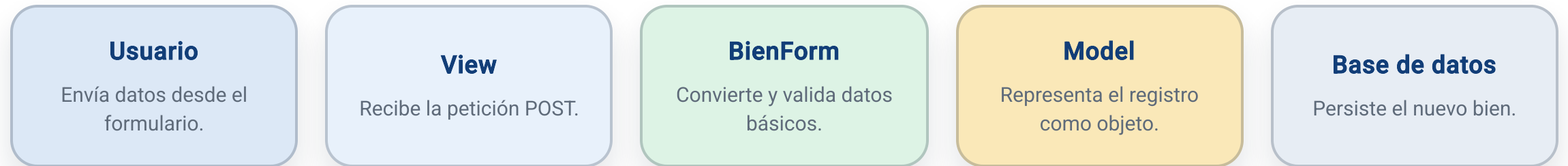
```
@login_required
def crear_bien(request):
    if request.method == "POST":
        form = BienForm(request.POST)

        if form.is_valid():
            form.save()
            return redirect("bienes:lista")
    else:
        form = BienForm()

    return render(request, "bienes/formulario.html", {"form": form})
```

Parte	Función
<code>request.method == "POST"</code>	Detecta envío del formulario.
<code>BienForm(request.POST)</code>	Construye formulario con datos recibidos.
<code>form.is_valid()</code>	Ejecuta validación básica del formulario.
<code>form.save()</code>	Guarda el registro en la base de datos.
<code>redirect()</code>	Envía al listado después de guardar.

Flujo de creación de un bien



En esta versión, la validación corresponde a reglas básicas del modelo y del formulario generado por Django.

Paso 9: actualizar listado de bienes

Editar:

```
apps/bienes/templates/bienes/lista.html
```

Contenido:

```
{% extends "base.html" %}

{% block title %}Bienes registrados{% endblock %}

{% block content %}
<h2>Bienes registrados</h2>

<p>
  <a href="{% url 'bienes:crear' %}">Registrar nuevo bien</a>
</p>

<table border="1">
  <thead>
    <tr>
      <th>Identificador</th>
      <th>Descripción</th>
      <th>Marca</th>
      <th>Valor</th>
      <th>Estatus</th>
    </tr>
  </thead>
```

Paso 10: completar listado de bienes

Continuar en `lista.html` .

```
<tbody>
  {% for bien in bienes %}
    <tr>
      <td>{{ bien.identificador }}</td>
      <td>{{ bien.descripcion }}</td>
      <td>{{ bien.marca }}</td>
      <td>{{ bien.valor }}</td>
      <td>{{ bien.estatus }}</td>
    </tr>
  {% empty %}
    <tr>
      <td colspan="5">No hay bienes registrados.</td>
    </tr>
  {% endfor %}
</tbody>
</table>
{% endblock %}
```

La plantilla no consulta la base de datos. Solo presenta la información que la vista envía mediante el contexto.

Paso 11: actualizar formulario de bienes

Editar:

```
apps/bienes/templates/bienes/formulario.html
```

Contenido:

```
{% extends "base.html" %}

{% block title %}Registrar bien{% endblock %}

{% block content %}
<h2>Registrar bien</h2>

<form method="post">
    {% csrf_token %}

    {{ form.non_field_errors }}

    {% for field in form %}
        <p>
            {{ field.label_tag }}<br>
            {{ field }}

            {% if field.errors %}
                <br>
                <strong>{{ field.errors }}</strong>
            {% endif %}
        </p>
    {% endfor %}
</form>
```

Paso 12: completar formulario de bienes

Continuar en `formulario.html` .

```
        <button type="submit">Guardar</button>
</form>

<p>
    <a href="{% url 'bienes:lista' %}">Volver al listado</a>
</p>
{% endblock %}
```

Elemento	Función
{% csrf_token %}	Protege la petición POST contra CSRF.
form.non_field_errors	Muestra errores generales del formulario.
{% for field in form %}	Recorre campos generados por BienForm .
field.errors	Muestra errores específicos del campo.

Paso 13: verificar rutas existentes

El archivo `apps/bienes/urls.py` se mantiene así:

```
from django.urls import path
from . import views

app_name = "bienes"

urlpatterns = [
    path("", views.lista_bienes, name="lista"),
    path("nuevo/", views.crear_bien, name="crear"),
]
```

Ruta	Vista	Propósito
<code>/bienes/</code>	<code>lista_bienes</code>	Muestra registros guardados.
<code>/bienes/nuevo/</code>	<code>crear_bien</code>	Captura un nuevo bien.

Paso 14: ejecutar revisión y migraciones

Ejecutar revisión:

```
python manage.py check
```

Crear migraciones:

```
python manage.py makemigrations
```

Aplicar migraciones:

```
python manage.py migrate
```

Levantar servidor:

```
python manage.py runserver
```

Cada vez que se modifica `models.py`, se debe revisar si corresponde generar y aplicar migraciones.

Paso 15: probar captura de bienes

Entrar al sistema:

```
http://127.0.0.1:8000/accounts/login/
```

Ir al listado:

```
http://127.0.0.1:8000/bienes/
```

Crear un bien:

```
http://127.0.0.1:8000/bienes/nuevo/
```


Ejemplo de prueba:

Campo	Valor
Identificador	b-0001-2026
Descripción	Laptop institucional
Marca	Dell
Valor	18500.00
Estatus	Bueno

Resultado esperado

Después de guardar:

- el usuario regresa al listado;
- el bien aparece en la tabla;
- el registro queda almacenado en SQLite;
- el registro también puede consultarse desde el administrador.

Validación en esta versión

En esta versión sí existe una validación básica, pero no personalizada.

Validación	Origen
Campo obligatorio	Definición del modelo y formulario.
Longitud máxima	<code>max_length</code> del modelo.
Tipo decimal	<code>DecimalField</code> .
Opciones de estatus	<code>choices</code> .
Identificador único	<code>unique=True</code> .

Estructura final de la versión 2

```
proyecto_seguro/  
  manage.py  
  requirements.txt  
  
  config/  
    settings.py  
    urls.py  
  
  apps/  
    __init__.py  
    bienes/  
      __init__.py  
      admin.py  
      apps.py  
      forms.py  
      models.py  
      urls.py  
      views.py  
      templates/  
        bienes/  
          lista.html  
          formulario.html  
  
  templates/  
    base.html  
    registration/  
      login.html
```

Lectura arquitectónica

Responsabilidad	Ubicación
Configuración global	config/settings.py
Rutas principales	config/urls.py
Rutas de bienes	apps/bienes/urls.py
Vistas protegidas	apps/bienes/views.py
Modelo de datos	apps/bienes/models.py
Formulario basado en modelo	apps/bienes/forms.py
Plantillas de bienes	apps/bienes/templates/bienes/
Login y base visual	templates/

La responsabilidad de persistencia aparece en esta versión mediante el modelo, el ORM y la base de datos.

Errores comunes

Error	Consecuencia
Modificar <code>models.py</code> y no ejecutar migraciones	La base de datos no refleja el modelo.
No registrar el modelo en <code>admin.py</code>	El modelo no aparece en el panel administrativo.
No importar <code>BienForm</code> en <code>views.py</code>	La vista de creación fallará.
No pasar <code>form</code> al template	La plantilla no podrá renderizar campos.
No pasar <code>bienes</code> al listado	La tabla no tendrá datos que mostrar.
Omitir <code>{% csrf_token %}</code>	El POST será rechazado o quedará inseguro.

Cierre de la versión 2

La segunda versión incorpora el modelo `Bien` y permite registrar información. El siguiente paso será agregar validaciones personalizadas y separar reglas reutilizables en una capa de seguridad.

En la versión 3 se incorporará:

- formato obligatorio para identificador;
- validación de año;
- validación de valores positivos;
- validación de texto;
- capa seguridad/ ;
- pruebas con casos válidos e inválidos.

Referencias de consulta

Recurso	Uso recomendado
Modelos en Django	Definición de entidades, campos y relaciones.
Migraciones	Creación y aplicación de cambios en base de datos.
ModelForm	Construcción de formularios a partir de modelos.
Administrador	Gestión interna de registros.
Templates	Presentación de listados y formularios.

Gracias
¿Preguntas?

